

# SRE Interview Questions & Answers Part 5

Q1. What happens internally  
when a Kubernetes pod gets  
OOMKilled?

OOMKilled happens when a process inside a container exceeds its memory limit, which is enforced by Linux cgroups.

When you define a memory limit in Kubernetes, the container runtime creates a cgroup for that container. A cgroup (control group) is a Linux kernel feature that isolates and limits resources like memory and CPU for a group of processes. This cgroup sets a hard memory boundary for all processes running inside the container. If the application tries to allocate memory beyond this limit, the Linux kernel detects that the cgroup has exceeded its allowed memory. At that point, the kernel invokes the OOM killer, which is a built-in mechanism that selects a process to terminate in order to reclaim memory. It uses factors like memory usage and OOM score to decide which process to kill, and typically it kills the main application process inside the container.

The process is terminated using SIGKILL, which cannot be caught or handled, so the application stops immediately. This causes the container to exit with code 137. The container runtime reports this to the kubelet, which then updates the pod status to OOMKilled and restarts the container based on the configured restart policy.

By default, Kubernetes uses a restart policy of **Always**, which means the kubelet restarts the **container inside the Pod**, not the Pod itself. If the issue persists, the container may restart repeatedly and the Pod can enter a `CrashLoopBackOff` state.

Q2. How do you handle Terraform state file locking and consistency in a large team?

In large teams, I maintain Terraform state consistency by using a remote backend like S3 with DynamoDB for state locking, which ensures only one operation can modify the state at a time.

I avoid concurrent applies by enforcing CI/CD-based execution, where all Terraform runs go through a centralized pipeline triggered via pull requests. This creates a single-writer model, adds approval gates, and provides auditability, preventing multiple engineers from modifying state simultaneously.

To reduce lock contention and blast radius, I split state files by environment or service, for example separate states for networking, EKS, and applications across dev, staging, and prod. This ensures teams can work independently without blocking each other and limits the impact of failures.

I also enable S3 versioning for recovery and handle stuck locks by inspecting the DynamoDB lock entry and using force-unlock carefully when needed.

Instead of keeping all infrastructure in one big Terraform state file, I split it into smaller state files based on environment or service. For example, I keep separate state files for networking, Kubernetes (EKS), and applications, and also separate them across dev, staging, and production.

This helps because only one person can modify a state file at a time due to locking. If everything is in one file, teams will block each other. But when state is split, different teams can work independently on their own parts without waiting.

It also reduces risk. If something goes wrong in one state file, it only affects that specific part instead of the entire infrastructure. This makes changes safer, faster, and easier to manage.

```
terraform {  
  backend "s3" {  
    bucket      = "tf-state"  
    key         = "prod/infra.tfstate"  
    region      = "ap-southeast-1"  
    dynamodb_table = "tf-state-locking"  
  }  
}
```

Q3. What are the "Four Golden Signals" you would monitor for a high-traffic API as an SRE?

The Four Golden Signals are latency, traffic, errors, and saturation. These are the most important metrics to monitor for any high-traffic API because they give a complete picture of system health.

Latency tells us how fast the API is responding. Instead of just looking at averages, we focus on percentiles like p95 or p99, which show how slow the experience is for real users. For example, if your API usually responds in 100ms but p99 latency goes to 2 seconds, some users are having a very poor experience.

Traffic tells us how many requests the system is handling, usually measured as requests per second. This helps us understand load. For example, during a sale or peak hours, traffic may suddenly increase, putting pressure on the system.

Errors represent failed requests, such as HTTP 5xx responses or timeouts. A rising error rate is a direct signal that something is broken. For example, if your error rate jumps from 0.1% to 5%, it likely indicates a deployment issue or a failing dependency.

Saturation shows how close your system is to its capacity. This includes metrics like CPU usage, memory usage, or database connections. For example, if CPU usage reaches 95%, the system is near its limit and may soon start slowing down or failing.

A real-world example: imagine your API receives a sudden spike in traffic during a product launch. As traffic increases, CPU usage (saturation) also increases. Once the system gets close to its limits, latency starts to rise because requests are taking longer to process. If the pressure continues, the system may begin to drop requests, leading to increased errors. By monitoring all four signals together, you can detect the issue early and scale your system before it impacts users.

Q4. How do you manage high cardinality in Prometheus metrics?

High cardinality in Prometheus happens when labels have too many unique values, which creates a very large number of time series. In Prometheus, every unique combination of metric name and labels becomes a separate time series, so if a label has unbounded values, the number of time series can grow very quickly and impact performance.

For example, if you define a metric like:

```
http_requests_total{user_id="123"}  
http_requests_total{user_id="456"}  
http_requests_total{user_id="789"}
```

and your system has millions of users, this will create millions of time series. This increases memory usage, slows down queries, and can even crash Prometheus.

To manage high cardinality, the most important step is to avoid using high-cardinality labels such as `user_id`, `session_id`, `request_id`, or IP address. Instead, we use stable and low-cardinality labels like `endpoint`, `HTTP method`, or `status code`.

For example:

```
http_requests_total{endpoint="/login", method="POST",  
status="200"}
```

This keeps the number of time series limited and meaningful.

Another approach is to use aggregation instead of tracking every individual value. Instead of tracking per user, we aggregate at the service or endpoint level using queries like:

```
sum(rate(http_requests_total[5m])) by (endpoint)
```

This reduces the amount of data and makes queries faster.

In production, I enforce labeling guidelines so developers don't introduce high-cardinality labels, and I treat metrics design as carefully as application code. In summary, I manage high cardinality by avoiding unbounded labels, aggregating metrics at a higher level, dropping unnecessary labels using relabeling, and continuously monitoring the number of time series to prevent performance issues.